← Project review - ML2_Supervised learning

Type of project	Individual
Duration	30 min
Passed Peer Reviews	1/3

Git project

ssh://git@git-ssh.21-school.ru:2222/students_repo/barle... Copy link Open

Student

barleysp level 10

About

Introduction

School 21 methodology only makes sense if peer-to-peer reviews are taken seriously. Please read all the guidelines carefully before starting the review.

- Please remain courteous, polite, respectful, and constructive in all communications during this review.

- Point out possible flaws in the person's work and take the time to discuss and debate them.
- Keep in mind that sometimes there may be differences in interpretation of tasks and scope of functionality. Please remain open to each other's vision.

Guidelines

- Evaluate only the files located in the src folder of the student's or group's GIT repository.
- If you have not finished the project yet, it is compulsory to read the entire instruction before starting the review.
- Ensure to start reviewing a group project only when the team is present in full.
- Use special flags in the checklist to report, for example, "empty work" if the repository does not contain the student's (or group's) work in the src folder of the develop branch, or "cheat" in the case of cheating, or if the student (or group) is unable to explain their work at any time during the review, or if any of the items below are not met. However, except in cases of cheating, you are encouraged to continue reviewing the project to identify the problems that caused the situation so that they can be avoided in the next review.
- Double check that the GIT repository is the one corresponding to the student or group.
- Carefully check that nothing malicious has been used to fool you.
- In controversial cases, remember that the checklist determines only the general order of the check. The final decision on project evaluation remains with the reviewer.

Main part

same as those attached below.

- 1.a. Analytical solution for the regression task: $\boldsymbol{\theta} = \left(X^T X\right)^{-1} X^T y$
- 1.b. Additional term in the loss function as R(w):
  $\min_{\boldsymbol{\theta}} \sum_{i=1}^{N} L\left(f(x_i, \boldsymbol{\theta}), y_i\right) + \lambda R(\boldsymbol{\theta})$
- 1.c. The L1 regularization adds a penalty $R(\boldsymbol{\theta}) = \|\boldsymbol{\theta}\|_1 = \sum_{i=1}^{d} |\theta_i|$ to the loss function. Since each non-zero coefficient adds to the penalty, it forces weak features to have zero coefficients. Thus, L1 regularization produces sparse solutions and could be used for feature selection.

- 1.d. To fit nonlinear dependencies, you should use nonlinear approaches for features, such as fitting polynomial features, using log(), exp(), sin(), and so on.

Main part. Task 2

- 2.g. If you make all previous steps correctly you will get same most common features as shown below:

```
>>> cnt.most_common (21)
[('Elevator', 19059),
('Hardwood Floors', 17317),
('CatsAllowed', 17229),
('DogsAllowed', 16134),
('Doorman', 15353),
('Dishwasher', 15024),
('NoFee', 13369),
('LaundryinBuilding', 12090),
('FitnessCenter', 9717),
('Pre-War', 6762),
('LaundryinUnit', 6294),
('RoofDeck', 4774),
('Outdoor Space', 3816),
('DiningRoom', 3677),
('HighSpeedInternet', 3150),
('', 2313),
('Balcony', 2181),
('SwimmingPool', 2012),
('LaundryInBuilding', 1941),
('NewConstruction', 1859),
('Terrace', 1602)]
```

Main part. Task 3

- All 22 features are collected to the single dataset and split into train and test parts

Main part. Task 4

- Model metrics on the test part of the dataset for your implementations could be worse than the corresponding Sklearn model, but not greater than 10% (for R2 score your implementation should beat 0.4).

Main part. Task 5

- The metrics will not differ much (unless you use extremely large coefficients for regularization).

Main part. Task 6

- Examples are given of why and where function normalization is mandatory and vice versa.
- The formula of the MinMaxScaler method is given and a custom implementation of the method is presented. A comparison of the custom MinMaxScaler method with the one from the sklearn.preprocessing library is provided.
- The formula of the StandardScaler method is given and a custom implementation of the method is presented. A comparison of the custom method StandardScaler with the one from the sklearn library is provided.

Main part. Task 7

- Scalers only need to be fitted to the training data and then applied to both the training and test parts.
- Scalers would not affect model fitting from the perspective of a hot-encoded feature.
- 7.a. Check:

```
>>> result_RMSE
    |  model                      |  train        |  test
```

```
0   |  Linreg default              |  1004.745010  |  972.199516
1   |  Ridre default               |  1004.745017  |  972.196842
2   |  Lasso default               |  1004.920458  |  972.332690
3   |  ElasticNet default          |  1160.064797  |  1113.727847
4   |  Linear MinMaxScaler         |  1004.745010  |  972.199516
5   |  Ridre MinMaxScaler          |  1004.799505  |  972.080655
6   |  Lasso MinMaxScaler          |  1005.151020  |  972.074165
7   |  ElasticNet MinMaxScaler     |  1483.345552  |  1427.473707
8   |  Linear StandardScaler       |  1004.745010  |  972.199516
9   |  Ridre StandardScaler        |  1004.745010  |  972.198800
10  |  Lasso StandardScaler        |  1004.775194  |  972.202979
11  |  ElasticNet StandardScaler   |  1050.469075  |  1009.348577
12  |  Linreg Polynomial           |  1031.748047  |  42199.689406
13  |  Ridge Polynomial            |  1031.829595  |  30009.295577
14  |  Lasso Polynomial            |  1040.587001  |  1058.243211
15  |  ElasticNet Polynomial       |  1047.658588  |  1083.695577
16  |  Naive mean                  |  1600.548517  |  1539.899338
17  |  Naive median                |  1646.816452  |  1581.719104
```

- 7.b. Check:

```
>>> result_MAE
    |  model                       |  train        |  test
0   |  Linreg default              |  691.009352   |  679.814161
1   |  Ridre default               |  691.004946   |  679.809463
2   |  Lasso default               |  690.461632   |  679.440326
3   |  ElasticNet default          |  780.166423   |  765.818279
4   |  Linear MinMaxScaler         |  691.009352   |  679.814161
5   |  Ridre MinMaxScaler          |  691.180216   |  679.886174
6   |  Lasso MinMaxScaler          |  690.696799   |  679.444960
7   |  ElasticNet MinMaxScaler     |  1050.545527  |  1027.644645
8   |  Linear StandardScaler       |  691.009352   |  679.814161
9   |  Ridre StandardScaler        |  691.007991   |  679.812810
10  |  Lasso StandardScaler        |  690.702192   |  679.592026
11  |  ElasticNet StandardScaler   |  715.448082   |  702.760471
12  |  Linreg Polynomial           |  722.063967   |  1293.686018
13  |  Ridge Polynomial            |  722.183919   |  1081.601478
14  |  Lasso Polynomial            |  727.168374   |  723.490115
15  |  ElasticNet Polynomial       |  733.250316   |  730.362151
16  |  Naive mean                  |  1141.803771  |  1112.093933
17  |  Naive median                |  1087.862371  |  1065.271637
```

Main part. Task 8

- Overfitted models performance should be the worst one.

Main part. Task 9

- The results of the mean and median from the previous lesson are presented in the final dataframe.

Main part. Task 10

- 10.a. Your metrics may differ. It depends on sklearn version. Below you see approximately right metrics.

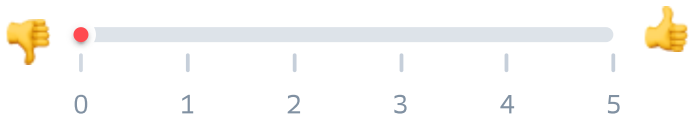
```
Model (MAE)                 |        train         |  test
Linreg default              |        691.0093519   |  679.8141606
Ridre default               |        691.0049463   |  679.8094625
Lasso default               |        690.4616316   |  679.4403263
ElasticNet default          |        780.166423    |  765.8182792
Linear MinMaxScaler         |        691.0093519   |  679.8141606
Ridre MinMaxScaler          |        691.1802158   |  679.8861745
Lasso MinMaxScaler          |        690.6967986   |  679.44496
ElasticNet MinMaxScaler     |        1050.545527   |  1027.644645
Linear StandardScaler       |        691.0093519   |  679.8141606
Ridre StandardScaler        |        691.0079907   |  679.8128099
Lasso StandardScaler        |        690.7021917   |  679.5920259
ElasticNet StandardScaler   |        715.4480823   |  702.7604708
Linreg Polynomial           |        722.0639668   |  1293.686004
Ridge Polynomial            |        722.1839185   |  1081.600658
Lasso Polynomial            |        727.1683736   |  723.4901151
ElasticNet Polynomial       |        733.2503162   |  730.3621506
Naive mean                  |       1141.803771    |  1112.093933
Naive median                |       1087.862371    |  1065.271637

Model (RMAE)                |        train         |  test
Linreg default              |        1004.74501    |   972.1995163
Ridre default               |        1004.745017   |   972.1968417
Lasso default               |        1004.920458   |   972.3326904
ElasticNet default          |        1160.064797   |   1113.727847
Linear MinMaxScaler         |        1004.74501    |   972.1995163
Ridre MinMaxScaler          |        1004.799505   |   972.0806554
Lasso MinMaxScaler          |        1005.15102    |   972.0741648
ElasticNet MinMaxScaler     |        1483.345552   |   1427.473707
Linear StandardScaler       |        1004.74501    |   972.1995163
Ridre StandardScaler        |        1004.74501    |   972.1988002
```

```
Lasso StandardScaler          |        1004.775194   |   972.2029794
ElasticNet StandardScaler     |        1050.469075   |   1009.348577
Linreg Polynomial             |        1031.748047   |   42199.68855
Ridge Polynomial              |        1031.829595   |   30009.23446
Lasso Polynomial              |        1040.587001   |   1058.243211
ElasticNet Polynomial         |        1047.658588   |   1083.695577
Naive mean                   |        1600.548517   |   1539.899338
Naive median                 |        1646.816452   |   1581.719104

Model (R2)                    |        train         |   test
Linreg default                |        0.605929406   |   0.60141024
Ridre default                 |        0.6059294     |   0.601412433
Lasso default                 |        0.605791769   |   0.601301033
ElasticNet default            |        0.474676524   |   0.476913523
Linear MinMaxScaler           |        0.605929406   |   0.60141024
Ridre MinMaxScaler            |        0.605886657   |   0.601507697
Lasso MinMaxScaler            |        0.605610859   |   0.601513019
ElasticNet MinMaxScaler       |        0.141091342   |   0.140686626
Linear StandardScaler         |        0.605929406   |   0.60141024
Ridre StandardScaler          |        0.605929405   |   0.601410828
Lasso StandardScaler          |        0.605905728   |   0.601407401
ElasticNet StandardScaler     |        0.569246459   |   0.570366947
Linreg Polynomial             |        0.584463072   |   -749.9894344
Ridge Polynomial              |        0.584397383   |   -378.7739476
Lasso Polynomial              |        0.577312791   |   0.527734395
ElasticNet Polynomial         |        0.571548302   |   0.504743782
Naive mean                   |        0.0           |   -0.00048481
Naive median                 |        -0.05900738   |   -0.00380357
```

- 10.b. We could see that the Lasso regression model with MinMaxScaler feature normalization method gives us more efficient results and also more stable. The difference between the training and the test sample metrics is only 0.00409784 on the R2 score.
- 10.c. We could see that the Lasso regression model with MinMaxScaler feature normalization method gives us more efficient results and also more stable. The difference between the training and the test sample metrics is only 0.00409784 on the R2 score.



Bonus part

Bonus part. Task 11

- 11.a. The best R2 performance on the test part > 0.61.
- 11.b. The best R2 performance on the test part > 0.61.
- 11.c. Algorithm is realized. The performance is comparable.

Yes No

Fails

Record of the online-review

Select a file or drag it here

Allowed file types are: .avi, .mov, .mp4, .webm and file size: max 800 MiB

Comment

Leave a comment...

Review